

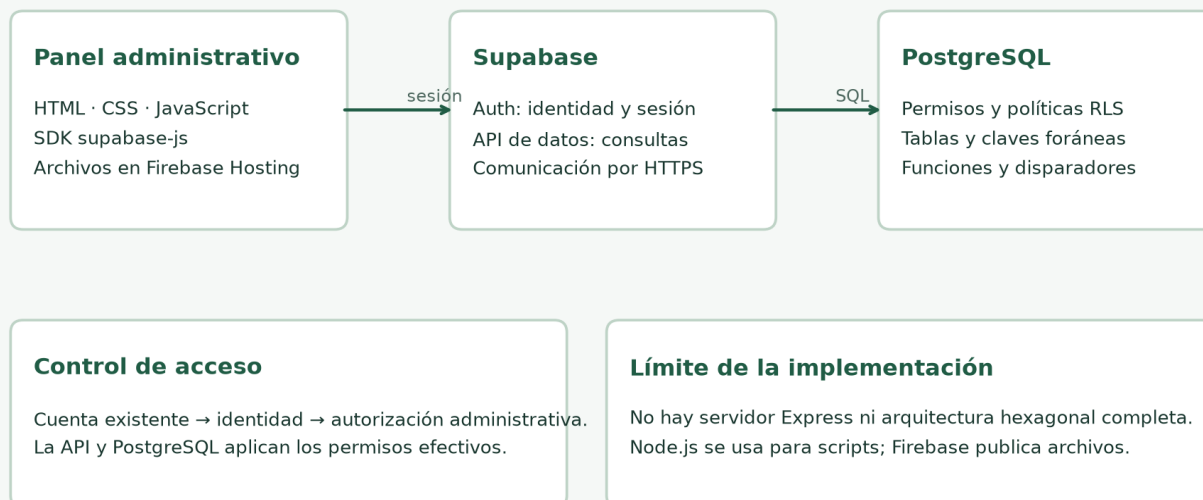
ALERTA SATIPO

Backend y persistencia de datos

Supabase · PostgreSQL · JavaScript

01 / Arquitectura del backend

Implementación actual: frontend estático y servicios administrados de Supabase



Documento de implementación · Versión revisada · 24 de septiembre de 2026

Alcance: panel administrativo, 300 lecturas históricas y operación demostrativa vinculada. Elaborado a partir del código del proyecto.

2.1. Objetivo y alcance

El backend almacena las lecturas históricas, autentica a los usuarios y controla el acceso a los datos. Permite consultar reportes e incidentes y actualizar estados de casos y tareas de demostración. La página publicada es exclusiva para administradores previamente autorizados, sin registro público. Las tablas de clientes y las funciones móviles permanecen en el esquema, pero el componente móvil no está incluido en el sitio publicado.

2.2. Tecnología utilizada

- Supabase: backend administrado con autenticación, API de datos y PostgreSQL.
- SQL y PL/pgSQL: tablas, relaciones, restricciones, políticas, funciones y disparadores.
- JavaScript y @supabase/supabase-js: integración de la interfaz con el backend.
- Node.js 22 o superior y npm: scripts de configuración, importación y compilación.
- Python: preparación de la muestra de 300 registros.
- HTML, CSS y JavaScript: frontend publicado en Firebase Hosting.

No existe un servidor propio con Express, Spring Boot o .NET. Node.js ejecuta herramientas del proyecto; no atiende las solicitudes del panel en producción. Firebase aloja la página y Supabase administra la base de datos.

2.3. Entorno y proyecto base

Las dependencias están declaradas en package.json. La preparación local utiliza:

```
npm install
npm run configure
npm start
```

Antes de configurar, se definen estas variables en un archivo .env local:

```
SATIPO_MODE=supabase
SUPABASE_URL=https://ddfmooylklgwnrmlxqzc.supabase.co
SUPABASE_PUBLISHABLE_KEY=<clave_publica>
SUPABASE_SECRET_KEY=<clave_privada_solo_para_scripts>
```

scripts/configure.mjs genera config/public.json con la URL y la clave pública. La clave privada se utiliza únicamente en herramientas administrativas y no se publica en el navegador ni se incluye en el repositorio.

Archivos principales:

- supabase/migrations/: definición y evolución del esquema.
- shared/js/backend.js: cliente Supabase, identificación, protección del acceso y cierre de sesión. Se ejecuta en el navegador.
- shared/js/data.js: consultas de reportes, incidentes y configuración.
- shared/js/dataset.js: consulta de lecturas históricas.
- web/js/operations.js: nodos virtuales, casos y mantenimiento.
- scripts/: preparación de datos, importación y configuración.
- tests/: pruebas de base de datos y navegador.

2.4. Arquitectura implementada

01 / Arquitectura del backend

Implementación actual: frontend estático y servicios administrados de Supabase

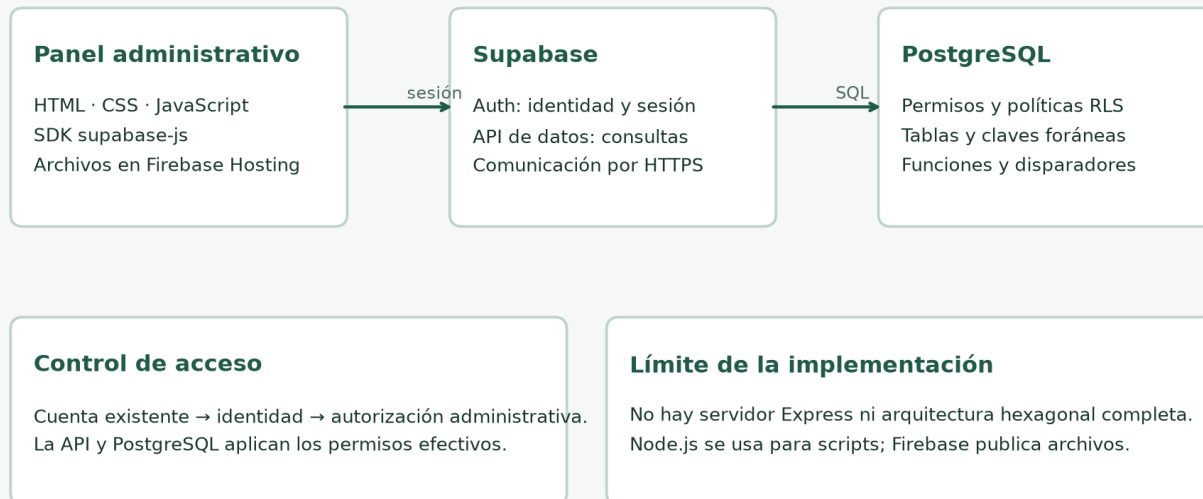


Figura 1. Arquitectura desplegada y responsabilidad de cada servicio.

```

Panel administrativo en el navegador
  | HTTPS + clave pública + token de sesión
  v
Supabase Auth / API de datos
  | Permisos, políticas RLS y funciones SQL
  v
PostgreSQL: tablas, relaciones y restricciones
  
```

Supabase Auth valida las credenciales y administra la sesión. El SDK envía las consultas a la API del proyecto. PostgreSQL aplica privilegios y políticas de seguridad por fila (RLS). La comprobación del navegador complementa estos controles, pero no los reemplaza.

Relación con la arquitectura hexagonal del ejemplo

El proyecto no implementa formalmente una arquitectura hexagonal con puertos, adaptadores y dominio independiente. Las entidades y restricciones están en SQL; la coordinación de consultas se encuentra en módulos JavaScript; la infraestructura es Supabase y su SDK. Esta distribución de responsabilidades no constituye por sí sola una arquitectura hexagonal.

Si ese patrón es obligatorio en la evaluación, queda por desarrollar un servicio propio, por ejemplo Node.js con Express: dominio con entidades y reglas; aplicación con casos de uso y puertos de repositorio; infraestructura con adaptadores Supabase y controladores HTTP. Esta ampliación es una propuesta pendiente, no una funcionalidad desplegada.

2.5. Modelo de datos y reglas

03 / Relaciones del módulo demostrativo

Extracto del modelo real: claves y cardinalidades de las cuatro tablas demo

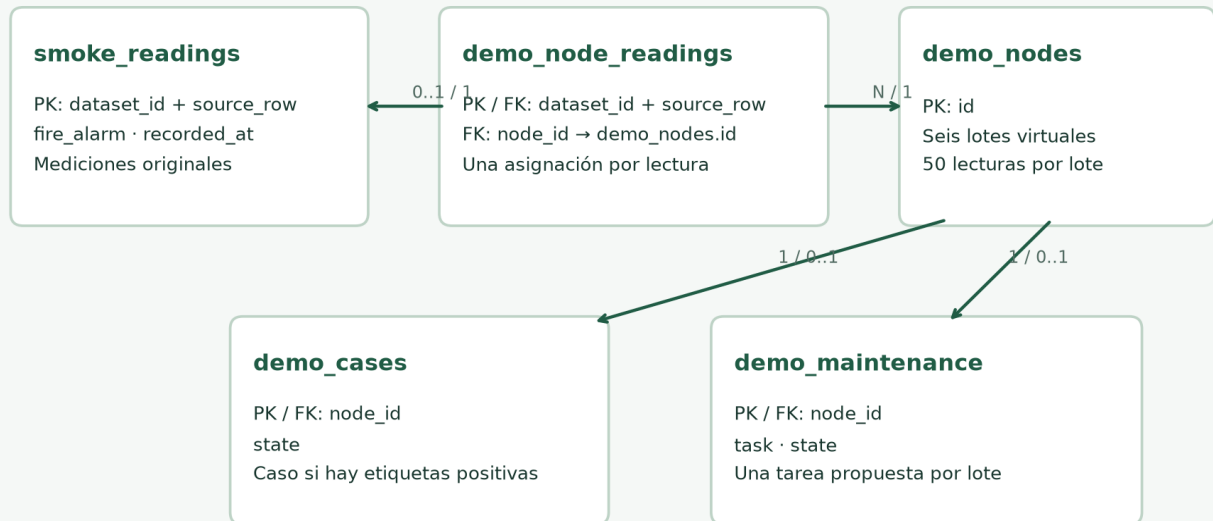


Figura 2. Relaciones del módulo demostrativo. Extracto; no incluye todas las tablas.

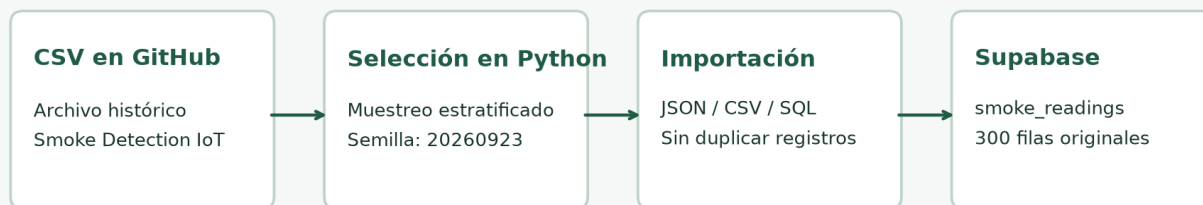
- clientes: perfiles vinculados a auth.users; también actúan como perfil base de administradores.
- administradores: identifica a los usuarios autorizados para administrar.
- smoke_readings: 300 lecturas con clave compuesta dataset_id y source_row.
- reportes: observaciones asociadas a un usuario.
- incidentes: incidentes vinculados a usuarios y, cuando corresponde, a reportes.
- incident_audit: historial de cambios de estado de incidentes.
- app_settings: umbrales de referencia del modelo demostrativo.
- demo_nodes: seis nodos virtuales, V-01 a V-06.
- demo_node_readings: relaciones entre nodos y lecturas mediante claves foráneas.
- demo_cases: casos de revisión de lotes con etiquetas positivas.
- demo_maintenance: tareas de mantenimiento demostrativo por nodo.

Los nodos son lotes de 50 lecturas ordenadas por source_row, no equipos instalados. El CSV no contiene ubicaciones, baterías, señal ni reparaciones. Un disparador crea un incidente al insertar un reporte y asigna una prioridad según la severidad reportada; ese número no es una probabilidad de incendio. Otro disparador registra cambios de estado en incident_audit. Los estados de las tablas demostrativas no tienen actualmente una auditoría equivalente.

2.6. Origen y carga de los datos

02 / Trazabilidad de las 300 lecturas

La fuente CSV se importa; el panel consulta la copia persistida en Supabase



6 nodos virtuales × 50 lecturas = 300 vínculos

Fire Alarm es una etiqueta histórica; no confirma incendios ni averías actuales.

Figura 3. Origen, selección y persistencia de las 300 lecturas.

Fuente: https://raw.githubusercontent.com/HamzaMa96/Smoke-Detection-IOT/main/smoke_detection_iot.csv

scripts/prepare_dataset.py descarga el CSV y selecciona una muestra aleatoria estratificada proporcional por Fire Alarm, con semilla 20260923. Genera JSON, CSV y SQL. La muestra contiene 214 etiquetas positivas y 86 negativas; no representa 214 incendios confirmados.

CSV de GitHub → selección en Python → 300 registros
→ importación → smoke_readings en Supabase

scripts/import-dataset.mjs utiliza esta operación:

```
await db.from('smoke_readings').upsert(rows, {
  onConflict: 'dataset_id,source_row',
  ignoreDuplicates: true
});
```

El importador verifica la cantidad y evita duplicados. No sustituye filas existentes y no sincroniza automáticamente los cambios del CSV. La aplicación consulta la copia de Supabase.

Para reproducir en un proyecto vacío: aplicar migración 001, migración 002, cargar las 300 filas mediante seed.sql o el importador y aplicar migración 003. Esta última requiere las 300 filas existentes. Las migraciones ya aplicadas no deben repetirse manualmente en el proyecto actual.

2.7. Conexión y API REST

URL del backend: <https://ddfmooylklgwnrmlxqzc.supabase.co>

`shared/js/backend.js` crea el cliente desde la configuración pública. Una consulta equivalente a la utilizada por el panel es:

```
const { data, error } = await client
  .from('smoke_readings')
  .select('*')
  .eq('dataset_id', 'smoke-detection-iot-300-v1')
  .order('source_row')
  .limit(300);
if (error) throw error;
```

Los endpoints pertenecen a la API de Supabase; no son rutas Express propias:

- POST `/auth/v1/token?grant_type=password`: inicio de sesión.
- GET `/rest/v1/smoke_readings`: lecturas históricas.
- GET `/rest/v1/demo_nodes`: nodos virtuales.
- GET `/rest/v1/demo_node_readings`: vínculos a las lecturas.
- GET `/rest/v1/demo_cases`: casos de revisión.
- PATCH `/rest/v1/demo_cases?node_id=eq.V-01`: estado de un caso.
- GET `/rest/v1/demo_maintenance`: tareas demostrativas.
- PATCH `/rest/v1/demo_maintenance?node_id=eq.V-01`: estado de una tarea.
- GET `/rest/v1/incidentes`: incidentes visibles para la sesión.
- PATCH `/rest/v1/incidentes?id=eq.<uuid>`: actualización autorizada.

El panel añade el estado anterior al filtro de actualización para detectar cambios concurrentes. Si la API devuelve cero filas, no se considera una modificación exitosa. PostgreSQL restringe los valores permitidos para cada estado.

2.8. Seguridad

El panel verifica la identidad mediante Supabase Auth y comprueba que exista una fila del usuario en administradores. El registro público está deshabilitado.

Las cuatro tablas demo solo permiten consultas a administradores. En demo_cases y demo_maintenance únicamente state es editable por administradores desde la API. La comprobación de permisos usa `private.is_admin()`.

El esquema original permite a usuarios autenticados consultar smoke_readings y app_settings. Reportes e incidentes aplican permisos por propietario o administrador. Por tanto, el acceso exclusivo al panel web no significa que todas las tablas tengan políticas exclusivamente administrativas. Las restricciones, los privilegios por columna y RLS se aplican también al llamar directamente a la API.

2.9. Procedimiento de pruebas con Postman

Esta sección es una guía reproducible, no evidencia de pruebas ya ejecutadas en Postman. Crear variables locales `base_url`, `publishable_key`, `admin_email`, `admin_password` y `access_token`. Utilizar una cuenta administrativa existente y no compartir contraseñas o tokens.

A. Autenticación

POST `{{base_url}}/auth/v1/token?grant_type=password`

Cabeceras: `apikey: {{publishable_key}}` y `Content-Type: application/json`.

```
{"email": "{{admin_email}}", "password": "{{admin_password}}"}"
```

Esperado: HTTP 200 y `access_token`. Guardar ese token en la variable local. Para las siguientes consultas agregar `apikey: {{publishable_key}}` y `Authorization: Bearer {{access_token}}`.

B. Consulta de lecturas

GET `{{base_url}}/rest/v1/smoke_readings?select=*&dataset_id=eq.smoke-detection-iot-300-v1&order=source_row.asc&limit=300`

Esperado: HTTP 200, 300 registros y 214 valores `fire_alarm=true`. Consultar `demo_nodes` y `demo_node_readings` con `select=*` debe devolver seis nodos y 300 vínculos, respectivamente.

C. Actualización controlada

Consultar primero GET `{{base_url}}/rest/v1/demo_maintenance?node_id=eq.V-01&select=*` y anotar el estado original. En una prueba controlada, realizar PATCH a la tabla filtrando `node_id` y el estado anterior. Agregar `Content-Type: application/json` y `Prefer: return=representation`.

```
{"state": "En progreso"}
```

Esperado: una fila actualizada. Volver a consultar para verificar persistencia y restaurar el estado original. Una respuesta sin filas requiere revisar permisos o concurrencia; no acredita un cambio exitoso.

D. Validación de permisos y restricciones

Un estado no permitido debe producir un error de restricción y no cambiar datos. Consultar una tabla `demo` sin token de usuario debe impedir el acceso. Una cuenta autenticada sin autorización administrativa debe recibir cero filas en las tablas `demo` por RLS. No hace falta crear cuentas para este documento: esta última condición también se verifica en las pruebas locales.

Adjuntar capturas propias de las solicitudes que se ejecuten, ocultando contraseña, tokens y claves privadas. Este documento no incluye capturas de Postman ni resultados ficticios.

2.10. Pruebas automatizadas y límites

`npm test` ejecuta `tests/database.test.mjs` sobre PostgreSQL embebido mediante PGlite. Verifica la muestra, aislamiento, privilegios, restricciones y auditoría. Es una prueba local, no una inspección completa de producción.

`python tests/panel_readings.py` compara las 300 filas con la fuente local y comprueba paginación, filtros, fallos de conexión y tamaños de pantalla.

`python tests/panel_operations.py` comprueba los vínculos, persistencia tras recarga, recuperación ante fallos y diseño adaptable. Modifica temporalmente estados demostrativos y los restaura; no crea usuarios.

No hay telemetría física, predicción validada de incendios ni historial real de reparaciones. Un servidor propio y una arquitectura hexagonal son ampliaciones pendientes si se exigen académicamente. El backend existente ofrece autenticación, persistencia, API y control de acceso mediante Supabase.

2.11. Referencias del proyecto

Panel: <https://alerta-satipo-76778920.web.app/>

Supabase: <https://supabase.com/dashboard/project/ddfmooylklgwnrmlxqzc>

Repositorio privado: <https://github.com/76778920-tech/alerta-satipo>

Evidencia de implementación: las tres migraciones SQL, `shared/js/backend.js`, `shared/js/data.js`, `shared/js/dataset.js`, `web/js/operations.js`, `scripts/prepare_dataset.py`, `scripts/import-dataset.mjs` y los archivos de pruebas citados. Los ejemplos del documento usan marcadores y no contienen credenciales reales.